TAREA GLOBAL: PROGRAMACIÓN ESTRUCTURADA EN PYTHON (UT2)

1. FICHA TÉCNICA DE LA ACTIVIDAD

Parámetro	Detalle
Módulo	Programación con Python (Optativa - ASIR)
Unidad	UT2. Programación Estructurada
Duración	13 horas estimadas
Herramientas	Visual Studio Code, Python 3, GNU/Linux
Formato	Archivo .zip con scripts .py y memoria técnica en PDF

2. DESCRIPCIÓN GENERAL Y OBJETIVOS

Esta tarea constituye el núcleo práctico de la unidad de Programación Estructurada. El objetivo es asentar las bases sintácticas del lenguaje Python, evolucionando desde estructuras secuenciales básicas hasta la implementación de lógica compleja mediante bifurcaciones condicionales y bucles de control, siempre bajo una óptica de **Administración de Sistemas y Redes (ASIR)**.

3. CUADERNO DE ACTIVIDADES

GRUPO A: EJERCICIOS INICIALES (Asimilación y Secuencialidad)

A.1. Secuenciales (Entrada, Proceso, Salida)

1. **Cálculo Geométrico:** Desarrolla un script que solicite la base y la altura de un rectángulo para calcular y mostrar su perímetro y área total.
2. **Aritmética Básica:** Crea un programa que pida dos números reales y realice las operaciones de suma, resta, multiplicación y división, presentando los resultados formateados.
3. **Conversor Térmico:** Escribe un script que reciba una temperatura en grados Fahrenheit y la transforme a su equivalente en grados Celsius aplicando la fórmula C = (F - 32) * 5/9.
4. **Media de Datos:** Diseña un programa que capture tres valores numéricos y devuelva la media aritmética ponderada de los mismos.

5. **Gestión de Tiempo:** Implementa un script que convierta una cantidad bruta de minutos introducida por el usuario en su desglose exacto de horas y minutos restantes.

6. **Cálculo de Nómina:** Programa un calculador de sueldo que sume al salario base un 10% de comisión sobre tres ventas realizadas, mostrando el total a percibir.

7. **Descuento Comercial:** Crea un script que aplique un descuento del 15% sobre el valor total de una compra solicitada por teclado e informe del precio final.

8. **Evaluación Académica:** Desarrolla un algoritmo que calcule la nota final de un alumno basándose en tres calificaciones parciales, un examen final y un trabajo práctico.

9. **Distancia Numérica:** Escribe un programa que determine la distancia absoluta (valor positivo) entre dos números cualesquiera introducidos por el usuario.

A.2. Condicionales y Validación

10. **Login Seguro:** Simula un sistema de autenticación solicitando usuario y contraseña (definidos previamente en variables). El script debe permitir un máximo de 3 intentos fallidos; si se exceden, debe mostrar un mensaje de "Acceso bloqueado por seguridad" y terminar la ejecución.

11. **Conversor de Moneda:** Crea un menú que permita convertir cantidades entre USD y EUR (tasa fija 0.95). El script debe validar que la cantidad ingresada sea un número positivo. Debe incluir un bucle que permita al usuario realizar múltiples conversiones seguidas y salir solo cuando elija una opción específica.

12. **Cálculo de IVA:** Diseña un programa que calcule el precio final de un producto. Solicita el precio base y el tipo de IVA (21%, 10% o 4%). El script debe validar mediante condicionales que el tipo introducido sea válido; si se introduce un valor distinto, debe mostrar un error específico y terminar o pedir el dato de nuevo.

13. **Rangos de Impares:** Solicita dos números enteros al usuario. Valida mediante condicionales que el primero sea menor que el segundo; si no es así, solicita los datos de nuevo o muestra un error. Si los datos son correctos, recorre el rango y muestra por pantalla únicamente los números impares presentes.

14. **Filtro de Positivos y Paridad:** Crea un sistema de entrada que solicite números indefinidamente. El programa solo procesará números de 3 o más dígitos (validación). Finalizará cuando se introduzca un número negativo o cero. Para cada número válido procesado, debe indicar si es par o impar.

GRUPO B: EJERCICIOS AVANZADOS (Lógica, Sistemas y Redes)

B.1. Procesamiento de Logs y Cadenas

15. **Extracción y Validación de IPs:** Desarrolla un script que reciba una cadena de texto representando un log de firewall con formato 'src=192.168.1.1 dst=10.0.0.5'. El programa debe usar métodos de slicing o split() para extraer las direcciones IP, validar que tienen cuatro octetos y mostrar un error si el formato es incorrecto.

16. **Análisis de Duración de Sesión:** Escribe un programa que reciba una cadena con el formato '[inicio: HH:MM:SS] [fin: HH:MM:SS]'. El script debe parsear las horas, minutos y segundos, calcular la diferencia exacta en segundos totales y devolver la duración de la sesión en un formato legible.

17. **Parseo de Syslog Estructurado:** Crea un script que reciba una línea de log estructurada: '[CRITICO] [AUTH] Login fallido - usuario admin'. El programa debe utilizar técnicas de búsqueda de subcadenas para separar el nivel de severidad, el servicio y el mensaje, e imprimir cada campo con una etiqueta clara para su posterior auditoría.

18. **Monitor de Transferencia de Datos:** Implementa un script que solicite el tamaño de un archivo en MB y la velocidad de la red en Mbps. El programa debe calcular el tiempo estimado de transferencia en segundos, validando que ambos valores sean positivos para evitar divisiones por cero.

19. **Calculador de SLA (Downtime):** Crea un programa que solicite el porcentaje de disponibilidad de un servicio (ej. 99.9%). El script debe calcular cuántas horas y minutos de inactividad (downtime) representa ese porcentaje en un mes de 30 días, informando si cumple con el estándar de alta disponibilidad (definido como < 45 minutos al mes).

20. **Simulador de Estado de Servicio:** Diseña un script que reciba el nombre de un servicio (ej. 'SSH', 'Apache', 'MySQL') y un código de estado (0=Down, 1=Up, 2=Mantenimiento). El programa debe usar una estructura condicional para imprimir el estado actual del servicio en formato amigable: 'Servicio [Nombre]: [Estado técnico]'.

B.2. Redes y Telemetría

21. **Cálculo de Hosts CIDR:** Implementa un algoritmo que, dada una máscara de subred en formato CIDR, calcule el número total de hosts útiles disponibles en dicha red.

22. **Analizador de Octetos:** Diseña un programa que valide una dirección IPv4 y descomponga cada octeto en su representación binaria de 8 bits y hexadecimal.

23. **Simulador de Jitter:** Escribe un script que reciba una serie de latencias (ping) en un array y calcule el Jitter de red (variación estadística de la latencia máxima menos la mínima).

B.3. Monitorización de Sistemas

24. **Umbrales de CPU:** Crea un simulador que monitorice la carga de CPU (%) y emita alertas visuales: Verde (Normal <= 50%), Amarillo (Aviso 51-85%) y Rojo (Crítico > 85%).

25. **Capacidad de Disco:** Desarrolla un script que calcule el espacio libre en disco tras proyectar el crecimiento de logs de sistema durante un periodo de 30 días.

26. **Rotación de Archivos:** Implementa la lógica de un sistema de rotación que renombre archivos de backup antiguos y gestione la cuota de almacenamiento máxima permitida.

(Nota: Estas actividades se corregirán gradualmente en las sesiones prácticas del aula para verificar la correcta asimilación e implementación de los algoritmos y del estilo de programación, algunas actividades formarán parte de la práctica correspondiente a la unidad).